Sistema de auditoría de transacciones

Proyecto Final de Desarrollo Web (UMG) — Red social de fotografías.

Una **auditoría** responde a: *¿quién hizo qué, cuándo y desde dónde?* Este documento describe qué evidencia deja hoy la aplicación y propone una tabla de auditoría para cerrar las brechas.

1. Transacciones de la aplicación

Transacción	Endpoint	Tablas que cambian
Registro de usuario	`POST /api/registrar/user`	`users`
Inicio de sesión	`POST /api/login/user`, `POST /api/login/google`	(ninguna; crea un token)
Actualizar perfil / contraseña	`PUT /api/actualizar/user`	`users`
Cambiar foto de perfil	`PUT /api/upload/user`	`users` (y el archivo de imagen)
Crear publicación	`POST /api/crear/publicacion`	`posts`
Subir imagen	`POST /api/subir-imagen/:id`	`posts`, `image_variants`, `face_detections`
Editar / borrar publicación	`PUT` / `DELETE` sobre `publicacion`	`posts`
Me gusta	`PUT /api/likes/publicacion`	`posts`
Seguir / dejar de seguir	`POST` / `DELETE /api/follow/...`	`followers`
Aplicar filtro a una foto	`POST /api/imagen/transformar`	`image_variants`, `posts` o `users`

2. Evidencia que existe hoy

Fuente	Qué registra	Dónde
Columnas de fecha	Cuándo se creó cada registro	`users.created_at`, `posts.fecha`, `followers.fecha_follow`, `image_variants.created_at`, `face_detections.created_at`
Logs de PM2	Salida y errores de la app, con fecha y hora	`logs/out.log`, `logs/error.log` (`time: true` en `ecosystem.config.js`)
Logs de acceso de nginx	IP, método, ruta, código de respuesta y hora de cada petición	`/var/log/nginx/access.log`
AWS CloudTrail	Acciones sobre la cuenta de AWS (crear/detener instancias, cambios de IAM, S3)	Consola de AWS
Restricciones de la BD	Bloquean duplicados y auto-seguimiento a nivel de base de datos	`UNIQUE`, `CHECK` en `database/schema.sql`

Limitaciones actuales

- La app **no guarda quién ejecutó cada acción** de forma estructurada: hay que cruzar logs de nginx con tablas.
- Los **borrados** (publicaciones, seguidores) no dejan rastro.
- Los cambios de perfil no guardan el valor anterior.
- Los logs de PM2 mezclan mensajes técnicos con eventos de negocio.

3. Qué debe registrarse

Evento	Importancia	Detalle a guardar
Inicio de sesión correcto y fallido	Alta	Usuario o correo intentado, IP, método (clásico o Google)
Registro de usuario	Alta	Usuario creado, IP
Cambio de contraseña o correo	Alta	Usuario, qué campo cambió (**nunca el valor**)
Crear / editar / borrar publicación	Media	Usuario, id del post
Subir imagen o aplicar filtro	Media	Usuario, id del post, filtro
Seguir / dejar de seguir	Baja	Usuario origen y destino
Acceso denegado (token inválido o vencido)	Alta	IP, ruta

Nunca guardar contraseñas, tokens JWT ni el contenido de los archivos en el registro.

4. Propuesta: tabla `audit_log`

```
        CREATE TABLE IF NOT EXISTS audit_log (
            id           BIGSERIAL PRIMARY KEY,
            ocurrio_en   TIMESTAMPTZ NOT NULL DEFAULT NOW(),
            user_id      INTEGER,                -- NULL si no hay sesión (login fallido)
            accion       VARCHAR(60)  NOT NULL,  -- p. ej. 'LOGIN_OK', 'POST_BORRADO'
            metodo       VARCHAR(10)  NOT NULL,  -- GET, POST, PUT, DELETE
            ruta         VARCHAR(255) NOT NULL,
            status_code  INTEGER      NOT NULL,
            ip           VARCHAR(45),
            detalle      JSONB DEFAULT '{}'
        );
        CREATE INDEX IF NOT EXISTS idx_audit_usuario ON audit_log(user_id, ocurrio_en
DESC);
        CREATE INDEX IF NOT EXISTS idx_audit_accion  ON audit_log(accion, ocurrio_en DESC);
```

`user_id` no es una clave foránea a propósito: el registro debe sobrevivir aunque el usuario se borre.

Middleware de auditoría (borrador)

```
        // middlewares/auditoria.js  (propuesta)
        const { sequelize } = require("../database/conexion");

        // Uso en rutas:  router.delete("/borrar/publicacion/:id", auth.auth,
auditar("POST_BORRADO"), ...)
        module.exports = (accion) => (req, res, next) => {
            res.on("finish", () => {
                sequelize.query(
                    `INSERT INTO audit_log (user_id, accion, metodo, ruta, status_code, ip,
detalle)
                     VALUES (:userId, :accion, :metodo, :ruta, :status, :ip, :detalle)`,
                    {
                        replacements: {
                            userId: req.user ? req.user.id : null,
                            accion,
                            metodo: req.method,
                            ruta: req.originalUrl,
                            status: res.statusCode,
                            ip: req.ip,
                            detalle: JSON.stringify({ params: req.params })
                        }
                    }
                ).catch((e) => console.error("No se pudo escribir la auditoría:",
e.message));
            });
            next();
        };
```

Dos detalles al implementarlo:

- Detrás de nginx, activar `app.set("trust proxy", true)` en `index.js`; si no, `req.ip` mostraría siempre `127.0.0.1`. nginx ya envía `X-Forwarded-For`.
- El registro se escribe al terminar la respuesta (`finish`), así un fallo de auditoría no rompe la petición del usuario.

5. Protección del registro de auditoría

- El usuario de la aplicación solo debe poder **insertar y leer**, nunca modificar o borrar:

```
REVOKE UPDATE, DELETE, TRUNCATE ON audit_log FROM app_user;
```

- Retención: 12 meses en la base; luego exportar a S3 y borrar de la base.
- Revisar semanalmente los inicios de sesión fallidos y los accesos denegados.

6. Consultas de ejemplo

```
-- Últimas acciones de un usuario
SELECT ocurrio_en, accion, ruta, status_code FROM audit_log
WHERE user_id = 5 ORDER BY ocurrio_en DESC LIMIT 50;

-- IPs con más inicios de sesión fallidos en las últimas 24 h
SELECT ip, COUNT(*) AS intentos FROM audit_log
WHERE accion = 'LOGIN_FALLIDO' AND ocurrio_en > NOW() - INTERVAL '24 hours'
GROUP BY ip ORDER BY intentos DESC;

-- Publicaciones borradas esta semana
SELECT ocurrio_en, user_id, detalle FROM audit_log
WHERE accion = 'POST_BORRADO' AND ocurrio_en > NOW() - INTERVAL '7 days';
```

7. Alternativa: auditoría dentro de PostgreSQL

La extensión **pgaudit** registra las sentencias SQL a nivel de motor, sin tocar el código de la app. Es más completa, pero está disponible en Amazon RDS y requiere instalación extra en una instancia propia; para este proyecto se recomienda la tabla `audit_log`.